المحاضرة — رحلة الأخطاء والاستثناءات

النطاق: كل ما يتعلق بالخطأ من لحظة حدوثه عند السيرفر لحد ما يظهر للمستخدم كنص عربي، ومكان الخلل في كل مرحلة، والمطلوب عمله في التاسك القادم. الكود المرجعي: بعد commit `b5c1d21` + التعديلات غير المرفوعة.

0) الفهرس

1. الصورة الكبيرة: 4 مفاهيم مختلفة بنسميهم كلهم "error"
2. خريطة الملفات ومسؤولية كل ملف
3. الرحلة الكاملة من ضغطة الزر لحد التوست
4. كل الأماكن اللي بيتولد فيها الخطأ (المصادر الفعلية)
5. الـ `guard_` — نقطة التحويل
6. الـ `FailureMapper` — جدول التحويل
7. شجرة الـ Failures
8. من Failure لـ State (والـ Equatable)
9. من Code لنص عربي (الترجمة)
10. مسارات الواجهة الثلاثة
11. الثقوب الحالية مرتبة بالأولوية

12.التصميم المقترح (بدون تنفيذ)

13.خطة التنفيذ خطوة بخطوة

14.قرارات وقواعد تاخدها معاك

1) الصورة الكبيرة: 4 حاجات بنسميهم كلهم "error"

أغلب اللخبطة بتحصل إن فيه 4 كيانات مختلفة، وكل واحد له مكان ومسؤولية. الخطأ إن أي واحد يعدّي حدود مرحلته.

#	الكيان	الطبقة	شكله	مين المسؤول عنه	مثال في كودك
1	**Exception**	Data	كلاس بيترمي بـ `throw`	الـ datasource / SDK	`AuthException('invalid_credentials')`
2	**Failure**	Domain	كلاس بيرجع في `Either.Left`	الـ repository	`AuthFailure(code: 'invalid_credentials')`
3	**State**	Presentation	كلاس sealed بيتبعت من البلوك	الـ Bloc	`SigninBlocError(failure: ...)`
4	**Translation Key**	Presentation	String زي `errors.auth.invalid_credentials`	الـ Mapper + json	`"الايميل أو الباسورد غلط"`

القاعدة الذهبية: Exception ميعبرش حدود الـ data. Failure ميعبرش حدود الـ presentation

الـ State مش شايل نص للمستخدم. النص بيتولد في آخر لحظة من ملف JSON

كل واحدة من دول ليها سبب وجود:

- **Exception** موجود لأن SDK بترمي، ومفيش طريقة تانية تتعامل بيها مع مكتبة مش بتاعتك.
- **Failure** موجودة عشان الـ domain ميعرفش Supabase. لو بكرة استبدلنا Supabase بـ REST API، الـ Failures تفضل زي ما هي.
- **State** موجود عشان الواجهة تتعامل مع حالات محدودة معروفة (closed set) بدل boolean متشعبة.
- **Key** موجود عشان اللغة. لو كتبت النص العربي جوه الـ Bloc، مش هتقدر تعمل إنجليزي أبدًا.

2) خريطة الملفات

الملف	مسؤوليته في رحلة الخطأ	بيتعامل مع
`features/auth/data/datasources/auth_remote_data_source_impl.dart`	بيضرب Supabase؛ هو المصدر رقم 1 للأخطاء. بيرمي `AuthException` من SDK أو من كودك.	Exception
`features/characters/data/datasources/character_datasource_impl.dart`	بيسحب `characters` من الداتابيز. مبيرميش حاجة بنفسه — أي خطأ جاي من	Exception

	SDK أو من الـ parsing.	
`core/errors/failure_mapper.dart`	الترجمة: Exception → Failure. أهم ملف في الموضوع كله.	Exception → Failure
`core/errors/failures.dart`	تعريف عيلة الـ Failure (7 كلاسات).	Failure
`features/auth/data/repositories/auth_repository_impl.dart`	بيعمل الحدود: `guard_` بيمسك أي exception ويرجّع `Left(Failure)`.	Exception → Failure
`features/characters/data/repositories/character_repository_impl.dart`	نسخة قديمة من نفس الفكرة (try/catch يدوي) — لسه مش موحّدة.	Exception → Failure
`core/usecase/base_usecase.dart`	بيعرّف إن كل usecase بيرجع `<<Future<Either<Failure, R` — يعني نوع الإرجاع مفروض على الكل.	Failure
`Auth Bloc/states/auth_states.dart/...`	`AuthError(message)` — لاحظ: String بس، مش Failure.	State
`SignIn Bloc/states/sign_in_states.dart/...`	`SigninBlocError(message, failure)`.	State
`SignUp/... Bloc/states/sign_up_bloc_states.dart`	`SignUpError(message, failure)`.	State

`characters/presentation/states//...character_states.dart`	State `.CharacterIError(message, failure)`	
`core/errors/app_error_mapper.dart`	Failure → translation key (title + .description)	Failure → Key
`assets/translations/ar.json`, `en.json`	مفاتيح `*.errors` بالنص النهائي.	Key → نص
`sign_in_screen.dart`, `sign_up_screen.dart`, `character_choice_screen.dart`	اللي بيعرض فعليًا (toast / retry / ولا حاجة).	State → UI

3) الرحلة الكاملة

السيناريو: المستخدم كتب باسورد غلط وضغط "ادخل الآن".

```
[1] SignInScreen.onTap setState(validate) -> [2] لو فيه خطأ وقف (مش بيوصل للسيرفر)
context.read<SignInBloc>().add(SignInEvent(params)) | v [3] SignInBloc: emit(SigninBlocLoading) | v
[4] SigninWithEmailUsecase.call(params) -> authRepository.signInWithEmail(params) // domain interface
| v [5] AuthRepositoryImpl.signInWithEmail = _guard(() => datasource.signInWithEmail(params)) | v [6]
AuthRemoteDataSourceImpl.signInWithEmail supabase.auth.signInWithPassword(...) --> HTTP 400 ^ | throw
AuthException('Invalid login credentials', | statusCode:'400', code:'invalid_credentials') | [7]
_guard.catch(e) -> return Left(FailureMapper.map(e)) FailureMapper: AuthException() =>
AuthFailure(message:..., code:'invalid_credentials') | v [8] SignInBloc: res.fold( (l) =>
emit(SigninBlocError(message: l.message, failure: l)), (r) => emit(SigninBlocSuccess(user: r))) | v
[9] BlocConsumer.listener في SignInScreen AppErrorTitleMapper.toKey(state.failure) ->
'errors.auth.invalid_credentials' .tr() -> "الايميل أو الباسورد غلط" AppToast.show(...) ->
toastification | v [10] المستخدم شاف الرسالة
```

لاحظ إن **الـ Bloc مبيعرفش** **Supabase**، والـ **Screen ميعرفش** **Failure** (بيسأل mapper)، والـ **json مالوش علاقة بالسيرفر** كل طبقة بتترجم لمفهوم الطبقة اللي بعدها بس.

نفس الرحلة لما الخطأ يكون من عندك إنت (مش من السيرفر)

```
// في الـ datasource
final user = res.user;
if (user == null) {
  throw AuthException('invalid_credentials', code: 'invalid_credentials');
}
// SDK من exception بس النتيجة فاضية -> مفيش (HTTP 200) "العملية نجحت" //
// فإنت اللي بترمي، وباقي الرحلة زي ما هي.
```

4) كل الأماكن اللي بيتولد فيها الخطأ

دي القائمة الكاملة في الكود الحالي — 8 مصادر. مهم تعرف كل واحد عشان تعرف إيه اللي بيغطيه الـ mapper وإيه اللي بيفلت.

#	المكان	الشرط	اللي بيترمي	يوصل Failure إيه
1	`signInWithEmail`	`res.user == null` (بدون throw من SDK)	`AuthException(code:'invalid_credentials')`	AuthFailure
2	`signInWithEmail`	مفيش صف في `users` لنفس الـ id	`AuthException(code:'user_not_found')`	AuthFailure
3	`signUpWithEmail`	`res.user == null` بعد signUp	`AuthException(code:'signup_failed')`	AuthFailure **بكود مش موجود في الـ mapper**

4	`signUpWithEmail`	Google: `currentUser == null` `currentSession`	`AuthException(code:'user_not_found')`	AuthFailure
5	`signUpWithEmail`	`== null` (تأكيد إيميل مطلوب)	`AuthException(code:'email_not_confirmed')`	AuthFailure
6	`getUserById`	مفيش صف users	`AuthException(code:'user_not_found')`	AuthFailure
7	SDK نفسه	أي فشل شبكة / صلاحيات / RLS / عمود مش موجود	`AuthException`, `PostgrestException`, `StorageException`, `FunctionException`, `SocketException`	حسب الـ mapper (فيه ثقوب — شوف §6)
8	`UserModel.fromJson` / `CharacterModel.fromJson`	عمود راجع `null` أو نوع مختلف (مثلاً `xp` راجع double)	`TypeError` / `FormatException`	ServerFailure (رسالة الـ cast)

المصدر رقم 8 هو أخطر واحد وأكتر واحد بيضيّع وقت. لو عمود في الداتابيز ناقص، الرسالة اللي بتوصل المستخدم "حدث خطأ، حاول مرة أخرى" — وإنت مش عارف السبب. الحل مش إنك تكتب `try/catch` في كل حتة، الحل إنك تحسّن الـ mapper (شوف §6). 11 نقطة.

5) الـ `_guard` — نقطة التحويل

```
Future<Either<Failure, T>> _guard<T>(Future<T> Function() call) async {
  try {
    return Right(await call());
  } catch (e) {
    return Left(FailureMapper.map(e));
  }
}

// كل دالة في الريبو سطر واحد:
@Override
Future<Either<Failure, UserEntity>> signInWithEmail(SigninParams params) =>
    _guard(() => authRemoteDataSource.signInWithEmail(params));
```

ليه `<T>` generic؟

عشان نفس الكود يخدم `UserEntity` و`void` و`bool` و`AuthUserResult`. من غير generic هتكتب 7 نسخ متطابقة.

ليه بنمسك `Exception` مش `Object`؟

لأن `catch (e)` بيمسك أي حاجة اترمت، حتى لو `Error` (زي `TypeError`). ولو كتبت `on Exception catch (e)`

هتفلت الأخطاء اللي بتيجي من الـ casting — وهي بالظبط اللي محتاجين نمسكها.

ليه مفيش rethrow؟

لأن عقد الريبو بيقول: بيرجع `Either`، ومش بيرمي. لو رميت برّه، الـ Bloc هيقع والـ `fold` مش هيتنفذ أبدًا.

الحالة الوحيدة اللي مسموح فيها بـ throw

الـ datasource — لأنه بيتكلم مع مكتبة خارجية عقدها throw. ومفيش try/catch فاضي حواليه، لأن ده كود ميت

(بيمسك ويرمي تاني نفس الحاجة).

6) الـ `FailureMapper` — جدول التحويل

```
abstract class FailureMapper {
  static Failure map(Object error) {
    return switch (error) {
      AuthException() => AuthFailure(
          message: error.toString(),
          code: error.code ?? 'unknown',
        ),
      PostgrestException() => ServerFailure(message: error.toString()),
      StorageException() => ServerFailure(message: error.toString()),
      SocketException() => NetworkFailure(message: error.toString()),
      _ => ServerFailure(message: error.toString()),
    };
  }
}
```

اللي شغال صح

- `switch` على النوع بـ (`AuthException()`) object pattern — بيمسك الـ subclasses كمان، زي `AuthApiException` و`AuthWeakPasswordException`، لأنهم `extends AuthException`.
- الترتيب: الحالات الأخص الأول. `_` في الآخر كـ fallback مضمون.
- مكان واحد بس — لو عايز تضيف نوع جديد تعدّل سطر واحد.

اللي ناقص (وهو اللي هنشتغل عليه)

#	النقص	الأثر العملي
1	**مفيش branch لـ `FunctionException`**	أي خطأ من Edge Function (زي `check-email-exists` وهو واقع) بيروح `_` → ServerFailure → "حدث خطأ". مش كارثة بس مش دقيق.
2	**مفيش branch لـ**	دي اللي بتترمي لما الشبكة تقع أثناء auth. هي subclass من

		`AuthException` → بتتحول **AuthFailure** مع `code: null`
	`AuthRetryableFetchException`	`''unknown`، بدل ما تتحول NetworkFailure. يعني "الشبكة واقعة" بتظهر "حدث خطأ".
3	**مفيش branch لـ `http.ClientException`**	على الموبايل أحيانًا الفشل بيطلع من طبقة http مش من Socket مباشرة. لو ممسكتهاش → ServerFailure.
4	**`PostgrestException.code` بيضيع**	الكود فيه معلومات ذهبية: `23505` = unique violation (إيميل متسجل)، `42501` = RLS رفض، `PGRST116` = صف مش موجود. دلوقتي كلهم "حدث خطأ".
5	**بيبلع الـ exception الأصلي**	مفيش `debugPrint` ولا log. لو المستخدم قال "بيظهرلي خطأ"، مفيش أي أثر تقدر تتتبعه.
6	**مفيش تمييز بين `TypeError` وبين فشل سيرفر**	غلطة في الـ schema بتظهر كأنها مشكلة شبكة. صعب جدًا تتشخص.

7) شجرة الـ Failures

```
Failure (abstract, Equatable)
├── ServerFailure       message           [مستخدم]
├── NetworkFailure      message           [مستخدم]
├── LocalFailure        message           [معرّف وغير مستخدم]
├── MatchMakingFailure  message           [معرّف وغير مستخدم]
├── BattleFailure       message           [معرّف وغير مستخدم]
├── AuthFailure         message + code    [مستخدم]
```

```
└── CacheFailure          message          [معرّف وغير مستخدم]
```

الكلاس	مين بيرجّعه دلوقتي	ملاحظة
`AuthFailure`	أي `AuthException`	الوحيد اللي شايل `code` — وده اللي بيخلي الترجمة ممكنة
`ServerFailure`	أي حاجة من Postgrest / Storage متعرفش	مفيش `code` → الترجمة بتقف عند "حدث خطأ"
`NetworkFailure`	فقط SocketException	نادرًا ما بيتفعّل (شوف §6 نقطة 3)
`LocalFailure` ... `CacheFailure`	ولا حد	مستقبلية — سيبهم

باج حقيقي في `Failure.props => [message]`**، و**`failures.dart`**:** `AuthFailure` **مش** بتعمل override يعني:

```
AuthFailure(message: 'x', code: 'user_not_found')
  == AuthFailure(message: 'x', code: 'email_not_confirmed')   // true !
```

السبب: الـ Equatable بتقارن `props`، والـ `code` مش موجود فيها. والنتيجة إن البلوك ممكن ميبعتش الحالة الجديدة، أو إن مقارنة حالتين مختلفتين تطلع متساوية. نفس الغلطة اللي كانت في `SigninBlocError` قبل ما تتصلح.

8) من Failure لـ State

الـ `fold` هو اللي بيقرر: لو `Left` يبقى error state، لو `Right` يبقى success.

```
res.fold(
  (l) => emit(SigninBlocError(message: l.message, failure: l)),
  (r) => emit(SigninBlocSuccess(user: r)),
);
```

البلوك	State الخطأ	شايل إيه	props
SignInBloc	`SigninBlocError`	message + failure	✅ `[message, failure]`
SignUpBloc	`SignUpError`	message + failure	✅ `[message, failure]`
CharacterBloc	`CharacterIError`	message + failure	✅ `[message, failure]`
AuthBloc	`AuthError`	**بس message**	⚠️ `[message]`

ليه `AuthError` مختلف؟

لأنه مش ناتج عن فورم معيّن، هو حالة عامة للتطبيق (sign out فشل مثلًا)، والـ router بيعتمد عليه: `AuthError() => AppRoutes.signin`. لكنه **مش شايل Failure** — يعني مستحيل تعرض رسالة دقيقة، ومستحيل تفرّق بين خطأ شبكة وخطأ صلاحيات.

9) من Code لنص عربي

```
class AppErrorTitleMapper {
  static String toKey(Failure failure) => switch (failure) {
```

```
    NetworkFailure() => 'errors.network',
    AuthFailure(:final code) => switch (code) {
      _ when code.contains('invalid_credentials') => 'errors.auth.invalid_credentials',
      _ when code.contains('email_not_confirmed') => 'errors.auth.email_not_confirmed',
      _ when code.contains('user_not_found') => 'errors.auth.user_not_found',
      _ => 'errors.unknown',
    },
    _ => 'errors.unknown',
  };
}
```

جدول التغطية الكامل

اللي حصل فعليًا	Failure	code	title key	العربي
باسورد غلط / إيميل غلط	AuthFailure	`invalid_credentials`	`errors.auth.invalid_credentials`	الايميل أو الباسورد غلط
الإيميل لسه متأكدش	AuthFailure	`email_not_confirmed`	`errors.auth.email_not_confirmed`	لم يتم تأكيد البريد الإلكتروني
Auth ناجح بس مفيش صف users	AuthFailure	`user_not_found`	`errors.auth.user_not_found`	الحساب غير موجود
signUp رجع	AuthFailure	`signup_failed`	**`errors.unknown`**	حدث خطأ،

user == null				حاول مرة أخرى
الإيميل متسجل قبل كده (من Supabase)	AuthFailure	user_already_exists	errors.unknown	حدث خطأ ... (مع إن عندنا key جاهز!)
باسورد ضعيف	AuthFailure	weak_password	errors.unknown	—
كتر محاولات	AuthFailure	over_request_rate_limit	errors.unknown	—
RLS رفضت / عمود ناقص / صف مش موجود	ServerFailure	—	errors.unknown	حدث خطأ ...
الشبكة واقعة (Socket)	NetworkFailure	—	errors.network	تحقق من اتصالك

بالإنترنت

❌ المفروض

الشبكة واقعة

AuthFailure `null → unknown` `errors.unknown` تكون

أثناء auth

network

Edge

Function ServerFailure — `errors.unknown` —

واقعة

عمود راجع

null

ServerFailure — `errors.unknown` —

(TypeError

)

الوصف (description) مش تغطية كاملة: `AppErrorDescMapper` بيرجّع key واحد بس (`invalid_credentials_sub = "تأكد وحاول تاني"`) و `null` لكل الباقي. فالتوست بيظهر بعنوان من غير سطر تحتاني في معظم الحالات.

ملاحظة على `code.contains(...)`

المقارنة بـ `contains` مش `==`. ميزتها إنها بتصمد لو الـ code جاي متبوع بحاجة. عيبها إنها مش دقيقة

(`'user_not_found'` و`'user_not_found_v2'` نفس الحاجة)، وأي code جديد مش مذكور بيروح `errors.unknown`

حتى لو عنده key جاهز في الـ json. الأنضف: `switch` على `code` بـ `==` أو `enum`.

10) مسارات الواجهة الثلاثة

أ) SignInScreen — `BlocConsumer`

```
if (state is SigninBlocError) {
  AppToast.show(
    title: AppErrorTitleMapper.toKey(state.failure).tr(),
    description: AppErrorDescMapper.toKey(state.failure)?.tr(),
    type: AppToastType.error,
  );
}
```

المسار ده سليم: failure → key → tr → toast. ملاحظة صغيرة: `AppErrorDescMapper.toKey(...)?.tr()` لو

رجعت `null` الناتج `null` و`AppToast` بيتعامل معاه (`description != null`) — تمام.

ب) SignUpScreen — `BlocListener`

نفس الفكرة مع `SignUpError`. زائد حالة منفصلة (`EmailIsExist`) بتعرض رسالة الإيميل المتسجل من **مفاتيح**

مباشرة مش من الـ mapper:

```
if (state is EmailIsExist && state.isExist) {
  AppToast.show(title: 'errors.auth.email_already_exists'.tr(), ...);
}
```

ده مقبول لأنه مش ناتج عن Failure أصلاً (الـ Edge Function بترجّع bool). بس يبقى في دماغك إن **فيه مسارين**

مختلفين لإظهار رسالة إيميل متسجل: واحد من الـ mapper (لو Supabase رفضت) وواحد مباشر (لو الـ Edge

Function قالت موجود).

ج) CharacterChoiceScreen — رسالة ثابتة

```
if (state is CharacterIError) {
  return _buildLoadFailure(context: context, message: "errors.unknown".tr());
}
```

المشكلة: الـ state شايل `failure` و`message`، والواجهة بتتجاهل الاتنين وتكتب نص ثابت. يعني أي خطأ في تحميل

الشخصيات — حتى لو شبكة — بيطلع "حدث خطأ".

د) AuthError — خطأ صامت

الـ router بيعمل redirect لـ `signin/` لما الحالة `AuthError` — ومفيش أي toast. حاليًا `AuthError` بيتولد في

حالتين: فشل sign out، وفشل فتح Google OAuth. يعني لو Google فشل، المستخدم بيرجع لشاشة الدخول **من**

غير أي تفسير. ده أخطر مسار UX في القائمة.

11) الثقوب الحالية مرتبة بالأولوية

#	الأولوية	الثقب	الأثر على المستخدم	الأثر عليك إنت (تطوير)
1	P0	`AuthFailure.props` ناقص `code`	—	states متساوية بالغلط، emit بيتوه، behavior غامض وصعب تتبعه

2	P0	الـ datasource بيستخدم `AuthException` لأخطاء من عندنا	—	خلط طبقات: انت بتستعير كلاس SDK ليعبّر عن حالة ملهاش علاقة بالـ SDK
3	P0	`AuthError` صامت + مش شايل Failure	فشل Google = رجوع صامت لشاشة الدخول	مفيش أي وسيلة تعرف سبب الفشل
4	P1	`signup_failed` → errors.unknown	"حدث خطأ" بدل رسالة مفهومة	—
5	P1	أكواد Supabase الجاهزة (`weak_password`, `user_already_exists`, `over_request_rate_limit`) مش مترجمة	رسائل عامة	wasted keys موجودة في json ومش مستخدمة
6	P1	`AuthRetryableFetchException` بتتحول AuthFailure بدل NetworkFailure	"حدث خطأ" بدل "اتصل بالنت"	تتبع الخطأ غلط
7	P1	`PostgrestException.code` بيضيع	رسائل عامة	مستحيل تفرّق RLS من unique violation من عمود ناقص

		`CharacterRepositoryImpl` بتحوّل كل حاجة لـ ServerFailure	"حدث خطأ" دايمًا	الكود نفسه فيه تعليق "that is temp"
8	P2			
9	P2	الواجهة بتتجاهل `CharacterIError.failure`	مفيش retry ذكي ولا رسالة صح	—
10	P2	مفيش logging للـ exception الأصلي	—	أصعب حاجة: مش عارف الخطأ جاي منين
11	P3	`TypeError` من `fromJson` بيتموّه كخطأ سيرفر	"حدث خطأ"	أخطاء الـ schema بتضيّع وقت
12	P3	4 كلاسات Failure مش مستخدمة	—	ضجيج في القراءة

12) التصميم المقترح (من غير تنفيذ)

12.1 استثناءات خاصة بالـ domain

الفكرة: الـ datasource يرمي **استثناء بيوصف الحالة**، مش يستعير كلاس SDK.

```
lib/core/errors/app_exceptions.dart

class ProfileMissingException implements Exception { final String userId; }
class EmailConfirmationRequiredException implements Exception {}
class AuthSignUpFailedException implements Exception {}
class DataParsingException implements Exception { final String source; }
```

الحالة الحالية	تبقى	ليه
`AuthException('invalid_credentials')` لما `res.user == null`	`InvalidCredentialsException`	الحالة دي معناها "البيانات غلط" مش "Supabase قالت كذا"
`AuthException('user_not_found')` لما مفيش صف	`ProfileMissingException(userId)`	مفيش صف في `users` ≠ المستخدم مش موجود في auth
`AuthException('signup_failed')`	`AuthSignUpFailedException`	عشان تقدر تديها code في الـ mapper
`AuthException('email_not_confirmed')`	`EmailConfirmationRequiredException`	الحالة دي مش خطأ من السيرفر، دي نتيجة (result) محتاجة تصرف مختلف

نقطة تصميم مهمة: `email_not_confirmed` أصلاً مش "خطأ" — هي نتيجة متوقعة. بدل ما ترميها كخطأ وترجع لشاشة الدخول، الأفضل (مستقبلًا) إن الريبو يرجّع `Result` منفصل ويودّيه على شاشة "فعّل إيميلك".

حاليًا أسيبها كـ `Failure` لحد ما نبني الشاشة.

12.2 الـ mapper بعد التعديل — الشكل المستهدف

```
static Failure map(Object error) => switch (error) {
  NetworkException()      => NetworkFailure(...),
  SocketException()       => NetworkFailure(...),
  ClientException()       => NetworkFailure(...),          // http
```

```
  AuthRetryableFetchException() => NetworkFailure(...),    // قبل AuthException

  ProfileMissingException()            => AuthFailure(code:'user_not_found'),
  EmailConfirmationRequiredException() => AuthFailure(code:'email_not_confirmed'),
  InvalidCredentialsException()        => AuthFailure(code:'invalid_credentials'),
  AuthSignUpFailedException()          => AuthFailure(code:'signup_failed'),

  FunctionException()     => ServerFailure(...),
  PostgrestException()    => ServerFailure(code: error.code, ...),
  StorageException()      => ServerFailure(...),
  TypeError()             => DataFailure(...),   // اختياري
  _                       => ServerFailure(...),
};
```

ترتيب الحالات مش رفاهية: `AuthRetryableFetchException` لازم تيجي **قبل** `AuthException` لأنها `extends AuthException` — نفس الكلام على `FunctionsFetchException` قبل `FunctionException` لو عكست الترتيب، الحالة الأخص مش هتتنفذ أبدًا.

12.3 الـ Failure بعد التعديل

```
class AuthFailure extends Failure {
  final String code;
  const AuthFailure({required super.message, required this.code});
  @override
  List<Object?> get props => [message, code];   // ← السطر الناقص
}

class ServerFailure extends Failure {
  final String? code;        // ← كود جديد: Postgres (23505 / 42501 / PGRST116)
  const ServerFailure({required super.message, this.code});
  @override
  List<Object?> get props => [message, code];
}
```

12.4 قاعدة القرار: throw ولا return؟

الحالة	throw	return Left

مكتبة خارجية رمت	سيبها تطلع	—
النتيجة رجعت فاضية/مستحيلة منطقيًا (user == null)	✅	—
التحقق من الداتا (validation)	❌	❌ (بيتعمل في الـ presentation)
حاجة متوقعة ومش خطأ (زي إيميل محتاج تأكيد)	❌	الأفضل Result مستقل

13) خطة التنفيذ خطوة بخطوة

كل خطوة ملف واحد + تحقق بعدها. الترتيب مرتّب عشان كل خطوة تبقى قابلة للتجربة لوحدها.

#	الملف	الخطوة	التحقق
1	`core/errors/failures.dart`	يضيف `AuthFailure.props` بيبني `code` بـ positional (كله named؟)	`flutter analyze` + تأكد إن مفيش place
2	`core/errors/app_exceptions.dart` (جديد)	4 كلاسات exceptions بتاعتنا	analyze

اكتب حالة اختبار ذهني:

3	`core/errors/failure_mapper.dart`	إضافة الحالات الجديدة **قبل** `AuthException`	كل exception → Failure المتوقع
4	`auth_remote_data_source_impl.dart`	استبدال 5 `throw AuthException` بالاستثناءات الجديدة	analyze + حاول sign in بباسورد غلط
5	`core/errors/app_error_mapper.dart`	إضافة `signup_failed` و`weak_password` و`user_already_exists` و`over_request_rate_limit`، وتغيير `contains` لـ `switch` على `==`	كل code له key موجود في ar/en
6	`assets/translations/*.json`	أي key جديد تضيفه (زي weak password / rate limit)	تشغيل التطبيق واللغة العربية + الإنجليزية
7	`character_repository_impl.dart`	يستخدم `_FailureMapper` + `guard` زي الـ auth	افصل النت وشوف الرسالة (لازم تبقى network)
8	`character_choice_screen.dart`	يستخدم `AppErrorTitleMapper` بدل النص الثابت	نفس test النت

			اعمل sign out وإنت
9	`auth_states.dart` + `auth_bloc.dart`	`AuthError` يشيل `Failure`؛ والجذر يعرض toast عليه (زي ما عملنا لتوست جوجل)	واصل النت ← لازم تشوف رسالة
10	`failure_mapper.dart`	إضافة `debugPrint` للـ exception الأصلي (في `kDebugMode` بس)	شوف الـ console

ترتيب مقترح للتنفيذ: 1 ← 2 ← 3 ← 4 ← 5 ← 6، وبعدها 7 ← 8. الخطوة 9 لوحدها لأنها بتلمس الـ router والـ UX. الخطوة 10 آخر حاجة.

14) قرارات وقواعد تاخدها معاك

السؤال	القرار	السبب
ليه مش `rethrow` وخلاص؟	مفيش `rethrow` في catch فاضي	كود ميت: بيمسك ويرمي نفس الحاجة. لو مش هتعمل حاجة، شيل الـ try/catch
هل أكتب نص للمستخدم في الـ data layer؟	لأ	الـ data ميعرفش لغة ولا شاشة. تكتب `code`، والـ presentation تترجم
String `code` ولا enum؟	String حاليًا، enum لاحقًا	enum بيمنع الأكواد الغلط وقت الـ compile. بس محتاج طبقة تحويل من كود Supabase لـ enum بتاعك

هل الـ Bloc يستخدم `failure.message` في الواجهة؟	لأ، ولا مرة تستخدم الـ key
هل نحتفظ بـ `message` في الـ Failure أصلًا؟	آه، للتشخيص والـ logs
هل كل failure لازم يبقى لها state؟	لأ `AuthFailure(code:'weak_password')`
إزاي أعرف إن الشبكة واقعة قبل ما أجرب؟	متختبرهاش بالتخمين

الـ message نص تقني (أحيانًا إنجليزي من السيرفر). الواجهة مش للعرض. وهي كمان بتدخل في `props` فبتفرّق الحالات

الـ state بيمثل حالات الواجهة الممكنة. مثلاً `invalid_credentials` و نفس الـ state، الرسالة بس مختلفة

افصل الـ wifi واعمل sign in. لو ظهر "حدث خطأ" يبقى الـ `AuthRetryableFetchException` مش متغطي

أسئلة تشخيص سريعة (تستخدمها وقت أي خطأ)

1. الشاشة عرضت إيه؟ لو "حدث خطأ، حاول مرة أخرى" ← يبقى الـ `errors.unknown`، يعني الـ code مش متغطي في `AppErrorTitleMapper`.
2. الـ state وصل ولا لأ؟ لو مفيش أي toast، يبقى إما البلوك مبعتهاش (props متساوية) أو الـ listener مش موجود.
3. الـ Failure نوعه إيه؟ ضيف `debugPrint('${failure.runtimeType}')` مؤقتًا.

4.لو `ServerFailure` → شوف الـ `PostgrestException` :log ولا `TypeError`؟

5.لو `AuthFailure(code: 'unknown')` → الـ SDK رمى `AuthException` من غير code (غالبًا شبكة أو session).

15) ملخص في 6 سطور

1.الخطأ بيتولد **Exception** في الـ data، ويتحول **Failure** في الـ repository عبر `guard` + _ `FailureMapper`.

2.الـ Bloc بيحوّل الـ Failure لـ **State** بـ `fold`، والواجهة بتحوّل الـ State لـ **key** عبر `AppErrorTitleMapper` ثم **نص** من الـ json.

3.الكود الحالي سليم في العمود الفقري، وناقص في: `AuthFailure.props`، واستعارة `AuthException`، وتغطية أكواد Supabase، وتفصيل `PostgrestException`.

4.أخطر ثقب `AuthError` :UX صامت — فشل Google بيرجّع المستخدم لشاشة الدخول من غير رسالة.

5.أخطر ثقب تطويري: مفيش logging للـ exception الأصلي، فكل خطأ بيبان "حدث خطأ" ومفيش أثر تتتبعه.

6.الخطة: failures → exceptions → mapper → datasource → error_mapper → json → character repo → character UI → AuthError → logging.